

JSI : Jurnal Sistem Informasi (*E-Journal*)

ISSN Print : 2085-1588

ISSN Online : 2355-4614

LINK : <https://jsi.ejournal.unsri.ac.id/index.php/jsi/index>

ANALISIS KETAHANAN LAYANAN WEB, DATABASE, DAN DNS TERHADAP SIMULASI FAULT INJECTION PADA SISTEM CONTAINER PROXMOX VE

Septa Rahmada Izania¹⁾, Vina Azahra²⁾, Mayyada Shafira³⁾, Leri Pasha Alpadilah⁴⁾,
Oktalib Ramadhan R⁵⁾, Basyopi Satria Ramadhan P. Astro⁶⁾

^{1,2,3,4,5,6}*Sistem Komputer, Fakultas Ilmu Komputer, Universitas Sriwijaya Indralaya*
^{1,2,3,4,5,6}*Jl. Raya Palembang-Prabumulih KM 32 Indralaya, Ogan Ilir, Sumatera Selatan, 30662*
Email: ¹*izaniaseptarahmada@gmail.com*, ²*ivinazahra@gmail.com*,
³*maysha.shafira@gmail.com*, ⁴*leripasha2006@gmail.com*, ⁵*okfidzrenatama123@gmail.com*,
⁶*basyopisatria04@gmail.com*

Abstrak

Ketahanan sistem menjadi aspek krusial dalam infrastruktur teknologi informasi *modern*, terutama dengan meningkatnya adopsi teknologi kontainerisasi untuk *deployment* aplikasi dan layanan. Penelitian ini mengeksplorasi pengaruh simulasi *fault injection* terhadap tiga komponen layanan kritis yaitu *DNS*, *web server*, dan *database* yang dijalankan dalam lingkungan *container* pada platform *Proxmox Virtual Environment (VE)*. Menggunakan pendekatan eksperimental, penelitian ini menerapkan berbagai *scenario* kegagalan termasuk *resource exhaustion*, *process termination*, dan *I/O disruption* yang berfungsi untuk mengevaluasi *respons* sistem terhadap kondisi anomali. Pengukuran difokuskan pada metrik primer yang mudah diamati tanpa alat eksternal, seperti: waktu *respons* halaman, status layanan, pemanfaatan CPU dan RAM *container*, serta pesan *error/log* aplikasi. Studi kasus ini menegaskan pentingnya desain redundansi (mis. replikasi DB, *DNS failover*), *caching*, dan pemantauan sumber daya secara proaktif pada lingkungan Proxmox VE.

Kata Kunci: *Proxmox VE, Container, Layanan, Respons.*

Abstract

System resilience has become a crucial aspect in modern information technology infrastructure, especially with the increasing adoption of containerization technology for application and service deployment. This study explores the effect of fault injection simulations on three critical service components: DNS, web server, and database running in a container environment on the Proxmox Virtual Environment (VE) platform. Using an experimental approach, this study implements various failure scenarios including resource exhaustion, process termination, and I/O disruption to evaluate the system's response to anomalous conditions. Measurements focus on primary

metrics that are easily observable without external tools, such as: page response time, service status, container CPU and RAM utilization, and application error/log messages. This case study emphasizes the importance of redundancy design (e.g., DB replication, DNS failover), caching, and proactive resource monitoring in the Proxmox VE environment.

Keywords: *Proxmox VE, Container, Service, Response.*

1. PENDAHULUAN

CPS (Cyber-Physical Systems) merupakan sistem kompleks yang mengintegrasikan proses fisik, komputasi, jaringan, serta Teknologi Informasi dan Komunikasi (TIK). Sistem semacam ini menuntut tingkat ketahanan yang tinggi karena berperan dalam mendukung infrastruktur dan layanan kritis. Ketahanan sistem didefinisikan sebagai kemampuan suatu sistem untuk mempertahankan tingkat kinerja yang dapat diterima dalam menghadapi berbagai gangguan, seperti kegagalan teknis, kesalahan manusia, maupun serangan siber [1]. Konsep ketahanan tersebut tidak hanya relevan pada CPS industri, tetapi juga pada infrastruktur teknologi informasi modern yang menopang layanan digital berbasis *web*, *database*, dan DNS. Dalam implementasinya, layanan-layanan tersebut kini banyak dijalankan di atas lingkungan komputasi terdistribusi dan terkontainerisasi, yang memberikan fleksibilitas tinggi namun juga menghadirkan tantangan baru dalam menjaga ketahanan layanan. Berbagai insiden siber dalam beberapa tahun terakhir menunjukkan bahwa kegagalan layanan TI dapat menimbulkan dampak signifikan, baik dari sisi operasional maupun keandalan layanan.

Teknologi kontainer telah berkembang sebagai solusi virtualisasi yang ringan dan fleksibel untuk mendukung penerapan aplikasi *modern*. Dibandingkan dengan mesin virtual tradisional, kontainer menawarkan *overhead* yang lebih rendah, waktu implementasi yang lebih cepat, serta portabilitas yang tinggi, sehingga banyak diadopsi dalam infrastruktur komputasi awan dan arsitektur layanan mikro [2]. Namun, mekanisme kontainer yang berbagi kernel dan sumber daya sistem juga menimbulkan tantangan, khususnya terkait isolasi sumber daya, manajemen performa, dan ketahanan layanan. Ketergantungan pada strategi pengelolaan sumber daya yang pada awalnya dirancang untuk mesin virtual dapat meningkatkan risiko degradasi performa maupun kegagalan layanan pada lingkungan terkontainerisasi. Oleh karena itu, pemahaman yang komprehensif terhadap karakteristik dan keterbatasan teknologi kontainer menjadi aspek penting dalam pengelolaan layanan-layanan kritis [3].

Berbagai studi menunjukkan bahwa kegagalan layanan jaringan bukan merupakan kondisi yang jarang terjadi dalam sistem berbasis *web*. Penelitian sebelumnya yang menganalisis kegagalan akses *web end-to-end* pada skala luas menemukan bahwa kegagalan akses terjadi secara konsisten, dengan sebagian besar kegagalan dipicu oleh permasalahan pada layanan DNS dan kegagalan pembentukan koneksi jaringan [4]. Selain itu, kegagalan layanan sering kali tidak

disebabkan oleh kerusakan *hardware*, melainkan oleh masalah konfigurasi sistem, gangguan jaringan, serta kondisi kelebihan beban sumber daya. Temuan-temuan tersebut menegaskan bahwa layanan *web* dan komponen pendukungnya, seperti DNS dan sistem *back end*, rentan terhadap berbagai jenis gangguan operasional yang dapat berdampak langsung pada ketersediaan dan performa layanan.

Fault injection merupakan metode pengujian yang banyak digunakan untuk mengevaluasi ketahanan layanan *software* dengan cara menyuntikkan gangguan buatan pada sistem yang sedang berjalan. Beberapa penelitian menunjukkan bahwa *fault injection* pada level aplikasi efektif untuk mengamati pengaruh kegagalan terhadap kinerja dan skalabilitas layanan berbasis *cloud*. Pendekatan ini memungkinkan evaluasi perilaku sistem ketika menghadapi kondisi abnormal seperti kelebihan beban sumber daya, penghentian proses layanan, maupun gangguan jaringan, sehingga memberikan gambaran yang lebih realistis terhadap tingkat ketahanan layanan [5]. Selain itu, berbagai metode toleransi kegagalan seperti replikasi, *restart*, dan *checkpointing* telah diusulkan dalam lingkungan *cloud*, namun efektivitasnya sangat bergantung pada karakteristik sistem dan jenis kegagalan yang terjadi [6].

Namun, sebagian besar penelitian sebelumnya berfokus pada lingkungan *cloud* berskala besar atau platform *cloud* publik, dengan asumsi ketersediaan sumber daya yang relatif melimpah dan mekanisme orkestrasi yang kompleks. Sementara itu, studi yang secara khusus mengevaluasi ketahanan layanan *web*, *database*, dan DNS dalam lingkungan *container* lokal berbasis Proxmox Virtual Environment (VE) masih terbatas. Padahal platform semacam ini banyak digunakan pada skenario skala kecil hingga menengah, seperti laboratorium, institusi pendidikan, dan infrastruktur privat, yang memiliki karakteristik dan keterbatasan sumber daya yang berbeda.

Berdasarkan latar belakang dan celah penelitian yang telah diuraikan, penelitian ini bertujuan untuk mengevaluasi pengaruh penerapan *fault injection* terhadap ketahanan layanan *web*, *database*, dan DNS yang dijalankan dalam lingkungan *container* berbasis Proxmox Virtual Environment (VE). Penelitian ini secara khusus memfokuskan evaluasi pada respons layanan terhadap berbagai skenario gangguan, seperti kelebihan beban sumber daya, gangguan jaringan, serta penghentian proses layanan. Kontribusi utama penelitian ini adalah penyediaan studi eksperimental yang merepresentasikan pengujian ketahanan layanan pada platform kontainerisasi lokal berskala kecil hingga menengah, serta penyajian hasil pengukuran sederhana yang mudah direproduksi tanpa ketergantungan pada alat pemantauan eksternal khusus.

2. METODE PENELITIAN

2.1 Desain Lingkungan dan Pendekatan Penelitian

Pada penelitian ini, digunakan pendekatan eksperimental dengan melakukan pengujian langsung pada layanan *web*, *database*, dan DNS yang dijalankan di dalam *container* pada platform Proxmox VE. Layanan ditempatkan pada *container* yang terpisah untuk memudahkan pemantauan dampak

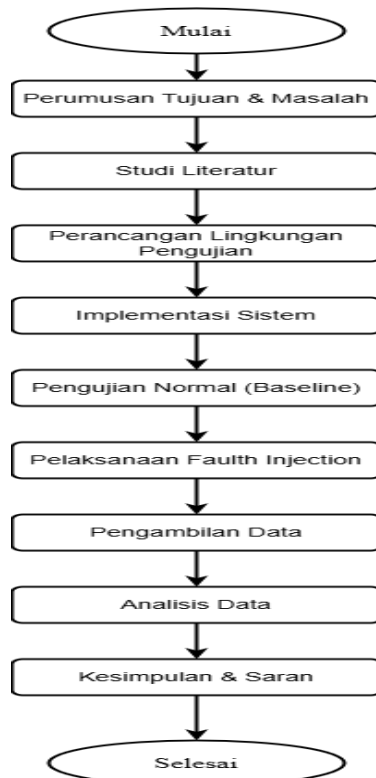
gangguan secara lebih spesifik. Pengujian dilakukan dalam lingkungan terkendali yaitu satu *node* Proxmox VE yang digunakan sebagai *host container*. Spesifikasi *hardware* dan konfigurasi *software* ditunjukkan sebagai berikut:

Tabel 1. Lingkungan Pengujian dan Spesifikasi Sistem

No	Container	Job	Sistem Operasi	Spesifikasi
1	DNS-CT	Resolusi nama domain	Debian 12	CPU 2 RAM 1 GB
2	WEB-CT	Aplikasi web server	Debian 12	CPU 3 RAM 2 GB
3	DB-CT	Database server	Debian 12	CPU 3 RAM 2 GB

Masing-masing *container* akan dikonfigurasi melalui *virtual bridge* bawaan Proxmox yang dimana komunikasi antar layanan berlangsung dalam jaringan internal. Arsitektur ini akan menghasilkan jalur berurutan yang dimana *web server* dikonfigurasi untuk mengakses database, sementara DNS menyediakan pemetaan domain untuk mengarahkan permintaan yang menuju *web server*.

2.2 Alur Penelitian



Gambar 1. Alur Perancangan

2.3 Skenario Pengujian

2.3.1 Pengujian Normal (*Baseline*)

Pada proses ini layanan diuji dalam kondisi operasi normal tanpa adanya gangguan atau injeksi kesalahan. Dalam fase ini, semua fungsi layanan dioperasikan secara murni dalam lingkungan yang tidak terpengaruh oleh interupsi atau simulasi gangguan buatan. Melakukan tes *baseline* merupakan langkah esensial untuk memetakan status awal yang seimbang dan fungsional dari infrastruktur. Keseimbangan ini nantinya akan dijadikan parameter utama untuk meninjau dan menganalisis seberapa besar perubahan atau degradasi kinerja yang terjadi setelah adanya *fault injection*. Sebelum melangkah lebih jauh, sangat penting untuk mengkonfirmasi ketersediaan dan fungsionalitas dari seluruh komponen utama, yaitu DNS, *web server*, dan *database*. Konfirmasi status ini dilakukan menggunakan *command line* spesifik yang dapat dilihat pada bagian berikut:

```
root@DNS:~# systemctl status bind9
* named.service - BIND Domain Name Server
   Loaded: loaded (/lib/systemd/system/named.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-12-08 03:48:54 UTC; 1h 24min ago
```

Gambar 2. Status Layanan DNS

```
root@DNS:~# sudo named-checkconf
root@DNS:~# sudo named-checkzone kelompok.dua /etc/bind/db.kelompok.dua
zone kelompok.dua/IN: loaded serial 6
OK
```

Gambar 3. Konfigurasi dan *Zone File*

```
root@DNS:~# top -b -nl | head -n 12
free -m
top - 05:33:45 up 1:44, 1 user, load average: 0.03, 0.10, 0.18
Tasks: 17 total, 1 running, 16 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 1024.0 total, 749.2 free, 44.5 used, 230.3 buff/cache
```

Gambar 4. Penggunaan Awal CPU dan RAM

Hasil Observasi pada layanan DNS menunjukkan bahwa layanan tersebut berada dalam kondisi fungsional yang prima. Hal ini terkonfirmasi dari status *check bind9* yang menunjukkan layanan berstatus berjalan dan cepat tanggap. Integritas sistem DNS semakin diperkuat oleh verifikasi sukses pada pengaturan dasar dan *zone file*, yang semuanya terdeteksi sah tanpa adanya kesalahan teknis. Dari segi manajemen sumber daya, DNS beroperasi dengan efisiensi tinggi: operasinya sangat tenang, hampir tidak menggunakan daya pemrosesan sentral (CPU), dan permintaan terhadap memori sistem (RAM) juga tercatat sangat rendah.

```
root@WEB:~# systemctl status apache2
* apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; preset: enab
   Active: active (running) since Mon 2025-12-08 03:48:58 UTC; 2h 0min ago
```

Gambar 5. Status Layanan *Web*

JSI : Jurnal Sistem Informasi (*E-Journal*)

ISSN Print : 2085-1588

ISSN Online : 2355-4614

LINK : <https://jsi.ejournal.unsri.ac.id/index.php/jsi/index>

```
root@WEB:~# curl -I http://localhost
HTTP/1.1 302 Found
Date: Mon, 08 Dec 2025 05:52:20 GMT
Server: Apache/2.4.65 (Debian)
Set-Cookie: PHPSESSID=263931ghrha4jofvdan0gd3qf8; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Location: login.php
Content-Type: text/html; charset=UTF-8
```

Gambar 6. Status *Service Web*

```
root@WEB:~# top -b -nl | head -n 12
free -m
top - 05:54:06 up 2:05, 1 user, load average: 0.07, 0.04, 0.06
Tasks: 23 total, 1 running, 22 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 33.3 sy, 0.0 ni, 66.7 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 2048.0 total, 1771.4 free, 32.8 used, 246.4 buff/cache
```

Gambar 7. Penggunaan Awal CPU dan RAM #2

Layanan *web server* menunjukkan kondisi aktif dari status *apache2* yang aktif serta pengecekan menggunakan *curl* juga menunjukkan respons *HTTP 302 Found*, menandakan bahwa *server* berfungsi dengan baik dan melakukan pengalihan ke halaman *login* sebagaimana mestinya. Kondisi stabil dengan beban CPU dan konsumsi memori yang sangat rendah.

```
* mariadb.service - MariaDB 10.11.14 database server
Loaded: loaded (/lib/systemd/system/mariadb.service; enabled; preset: en
Active: active (running) since Mon 2025-12-08 03:49:10 UTC; 2h 0min ago
```

Gambar 8. Status Layanan *Database*

```
root@DB:~# top -b -nl | head -n 12
top - 05:54:06 up 2:04, 1 user, load average: 0.07, 0.04, 0.06
Tasks: 17 total, 1 running, 16 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 0.0 id, 100.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 2048.0 total, 1693.8 free, 97.5 used, 256.8 buff/cache
```

Gambar 9. Penggunaan Awal CPU dan RAM #3

Layanan *database* berada dalam kondisi aktif dari status *mariadb service* yang berjalan normal tanpa pesan kesalahan. Pemeriksaan penggunaan sumber daya memperlihatkan bahwa beban CPU dan konsumsi RAM juga berada pada tingkat rendah.

2.3.2 Pengujian Fault Injection

Pada proses ini dilakukan simulasi gangguan untuk melihat ketahanan layanan DNS, *web server*, dan *database* ketika mengalami kondisi *abnormal*. Setiap gangguan dijalankan langsung pada *container target*, sedangkan dua *container* lainnya dibiarkan bekerja *normal* untuk melihat dampak spesifik terhadap tiap layanan. Berikut adalah *command* yang digunakan:


```
root@DNS:~# systemctl stop bind9
root@DNS:~# systemctl status bind9
* named.service - BIND Domain Name Server
   Loaded: loaded (/lib/systemd/system/named.service; enabled; preset: enabled)
   Active: inactive (dead) since Mon 2025-12-08 08:33:47 UTC; 23s ago
```

Gambar 10. Penghentian Layanan DNS

```
root@WEB:~# systemctl stop apache2
root@WEB:~# systemctl status apache2
* apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: inactive (dead) since Mon 2025-12-08 14:45:36 UTC; 18s ago
```

Gambar 11. Penghentian Layanan *Web*

```
root@WEB:~# stress --cpu 2 --timeout 60
stress: info: [491] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
```

Gambar 12. Pengujian *Overload* CPU Pada *Web*

```
root@WEB:~# stress --vm 1 --vm-bytes 512M --timeout 60
stress: FAIL: [549] (251) unrecognized option: --timeout 60
```

Gambar 13. Pengujian RAM *Exhaustion* Pada *Web*

```
root@DB:~# systemctl stop mariadb
root@DB:~# systemctl status mariadb
* mariadb.service - MariaDB 10.11.14 database server
   Loaded: loaded (/lib/systemd/system/mariadb.service; enabled; preset: enabled)
   Active: inactive (dead) since Mon 2025-12-08 15:55:36 UTC; 27s ago
```

Gambar 14. Penghentian Layanan *Database*

3. HASIL DAN ANALISIS

Dari pengujian yang dilakukan selama penelitian berlangsung didapat hasil berikut :

3.1 CT-DNS

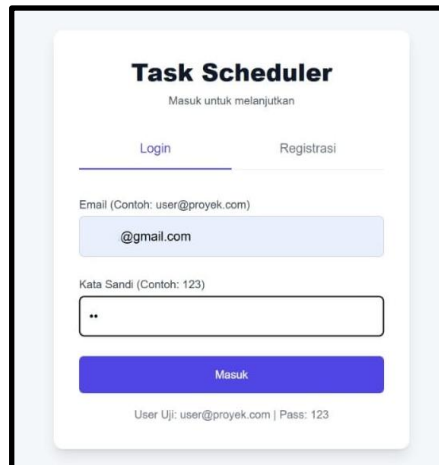
Pada pengujian *process termination* yang menghentikan layanan bind9, ditemukan fakta menarik bahwa aplikasi web masih bisa diakses secara normal. Akses yang tanpa hambatan ini disebabkan oleh cara kerja sistem resolusi nama: alamat IP server web tersebut sebelumnya telah tersimpan di dalam *cache* pada tingkatan resolusi yang lebih dekat ke *client*. Konsekuensinya, permintaan pertama kali untuk mengakses situs web tersebut masih bisa diarahkan ke tujuan yang tepat meskipun layanan DNS sedang dalam keadaan nonaktif. Di bawah ini adalah visualisasi antarmuka aplikasi web pada saat pengguna melakukan langkah masuk (*login*):

JSI : Jurnal Sistem Informasi (*E-Journal*)

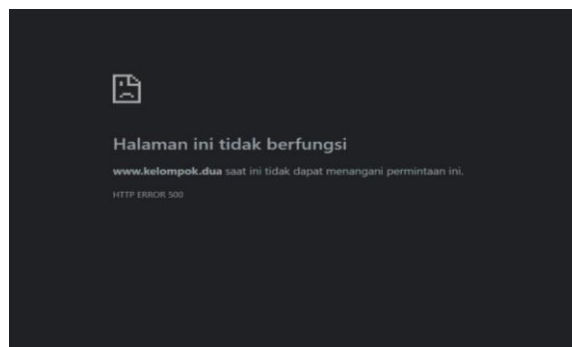
ISSN Print : 2085-1588

ISSN Online : 2355-4614

LINK : <https://jsi.ejournal.unsri.ac.id/index.php/jsi/index>



Gambar 15. Tampilan Halaman *Login*



Gambar 16. Tampilan Setelah *User Mencoba Login*

Ketika pengguna mencoba melakukan proses *login*, aplikasi menampilkan *HTTP ERROR 500*. Kondisi ini mengindikasikan bahwa komponen *backend* masih melakukan pemanggilan *hostname* secara internal untuk kebutuhan autentikasi atau pemrosesan sesi. Karena proses tersebut membutuhkan fungsi resolusi nama di sisi *server*, penghentian layanan DNS menyebabkan kegagalan pada tahap tersebut. Temuan ini menegaskan bahwa layanan *backend* tetap sensitif terhadap gangguan DNS meskipun akses awal pengguna tampak tidak terpengaruh.

3.2 CT-WEB

Pada pengujian dengan menghentikan layanan *apache2*, dampak yang ditimbulkan terhadap aplikasi web sangatlah instan dan signifikan. Aplikasi web target seketika mengalami kegagalan total (*error*).

Secara spesifik, setiap upaya akses yang dilakukan oleh pengguna melalui *browser* akan segera menghasilkan pesan kesalahan standar, yaitu '*502 Bad Gateway*'. Hal ini menunjukkan bahwa *proxy* atau *gateway* di depan web server tidak dapat berkomunikasi dengan *apache2* karena layanan tersebut tidak lagi berjalan. Kesimpulan dari pengujian ini adalah bahwa *web server*

(*apache2*) merupakan titik kritis dan tunggal (*Single Point of Failure*) bagi ketersediaan aplikasi, di mana penghentiannya secara langsung menghilangkan akses bagi pengguna.



Gambar 17. Tampilan Setelah *Stop Service*

Sedangkan pada pengujian *Overload CPU* dan *RAM Exhaustion*, tidak tampak gangguan berarti pada tampilan maupun fungsi aplikasi *web*. Kondisi ini menunjukkan bahwa beban yang diberikan belum mencapai tingkat kritis yang mampu menghambat operasi inti layanan. Dalam arsitektur *container Proxmox*, terdapat mekanisme manajemen sumber daya yang membuat proses layanan tetap memperoleh alokasi CPU dan memori minimum meskipun terjadi tekanan tinggi pada sistem. Selain itu, karakteristik aplikasi *web* yang diuji relatif ringan, sehingga tidak membutuhkan komputasi besar untuk memproses permintaan pengguna.

3.3 CT-DB

Pada pengujian penghentian layanan *database*, *MariaDB* dimatikan untuk melihat dampaknya terhadap aplikasi. Hasilnya, halaman awal *web* tetap dapat ditampilkan karena proses tersebut tidak memerlukan interaksi langsung dengan basis data. Namun, saat pengguna mencoba masuk ke sistem, aplikasi tidak dapat melakukan verifikasi kredensial sehingga muncul *HTTP ERROR 500*.

Hal ini menunjukkan bahwa fungsi-fungsi yang bergantung pada akses data bersifat sangat sensitif terhadap gangguan layanan *database*, sementara komponen antarmuka *web* masih dapat berjalan selama tidak memerlukan proses baca tulis ke basis data. Temuan ini sejalan dengan pola yang terlihat pada pengujian DNS, di mana gangguan pada layanan inti baru terasa pada bagian yang membutuhkan dependensi tersebut.

3.4 Keterbatasan Penelitian

Penelitian ini dilakukan pada lingkungan lokal dengan jumlah *container* yang terbatas, sehingga variasi beban dan kompleksitas arsitektur sistem tidak sepenuhnya mencerminkan kondisi produksi yang sesungguhnya. Selain itu, pengujian fault injection hanya mencakup beberapa jenis gangguan dasar seperti penghentian layanan (*process termination*), peningkatan beban CPU, dan konsumsi memori. Simulasi gangguan berbasis jaringan atau kerusakan fisik perangkat tidak dapat dilakukan karena keterbatasan akses dan alat pendukung.

Penelitian ini juga tidak menggunakan perangkat pemantauan eksternal atau *benchmarking tools* yang mampu mengukur *latensi*, *throughput*, maupun waktu *respons* secara presisi. Akibatnya, evaluasi kinerja lebih mengandalkan observasi langsung dan pembacaan sumber daya dari Proxmox, sehingga tingkat ketelitian pengukuran tidak setinggi metode berbasis instrumen otomatis.

Selama proses pengujian, beberapa layanan tetap dapat beroperasi secara normal meskipun berada dalam kondisi gangguan tertentu. Fenomena ini dipengaruhi oleh mekanisme internal sistem operasi dan *runtime container*, seperti penjadwalan proses, manajemen memori, serta isolasi sumber daya, yang mampu memberikan toleransi sementara terhadap tekanan beban. Mekanisme tersebut dapat menunda terjadinya kegagalan layanan yang teramati secara langsung, sehingga dampak *fault injection* pada *level* aplikasi tidak selalu muncul secara instan.

Keterbatasan-keterbatasan tersebut menyebabkan hasil penelitian ini lebih menggambarkan ketahanan sistem dalam skala terbatas dan lingkungan yang terkontrol. Meskipun demikian, temuan penelitian tetap memberikan gambaran awal yang relevan mengenai perilaku layanan *web*, *database*, dan DNS ketika menghadapi gangguan dasar. Hasil ini menunjukkan bahwa ketahanan layanan tidak hanya ditentukan oleh tekanan terhadap sumber daya, tetapi juga sangat bergantung pada keberlangsungan proses inti setiap layanan. Dengan demikian, penelitian ini dapat menjadi landasan awal untuk pengembangan studi lanjutan yang melibatkan skala sistem yang lebih besar, model gangguan yang lebih beragam, serta penerapan kerangka observabilitas yang lebih komprehensif sesuai dengan prinsip sistem toleran kesalahan dan praktik *Site Reliability Engineering*.

4. KESIMPULAN

Dengan penelitian ini dapat disimpulkan bahwa ketahanan layanan pada lingkungan container Proxmox VE sangat dipengaruhi oleh keberlangsungan proses seperti DNS, web server, dan database. Hasil pengujian menunjukkan bahwa meskipun tampilan awal aplikasi masih dapat diakses saat DNS atau database dimatikan, proses login dan fungsi backend langsung mengalami kegagalan. Sementara itu, penghentian web server menyebabkan layanan tidak dapat diakses sama sekali. Kondisi tekanan sumber daya seperti overload CPU dan RAM tidak memberikan dampak berarti karena adanya toleransi bawaan dari container dan ringan-nya aplikasi.

REFERENSI

- [1] Dedousis, P. (2023). Enhancing operational resilience of critical infrastructure processes through chaos engineering. IEEE Access, (Published 3 October 2023).
- [2] Adam S.Z.Belloum (2022). Containerization technologies: taxonomies, applications and challenges. Springer Nature, 78, 1144–1181. (Originally published 8 June 2021).

- [3] Aditya Bhardwaj (2021). Virtualization in cloud computing: Moving from hypervisor to containerization—A survey. Journal/Publisher.
- [4] Oppenheimer, D., Ganapathi, A., & Patterson, D. (2003). Why do Internet services fail, and what can be done about it? In USITS '03: Proceedings of the 4th USENIX Symposium on Internet Technologies and Systems (pp. 1–16).
- [5] Al-Said Ahmad (2022). Scalability resilience framework using application-level fault injection for cloud-based software services. *Journal Name*, 11(1). (Published 3 January 2022).
- [6] Muhammad Asim Shahid (2021). Towards resilient method: An exhaustive survey of fault tolerance methods in the cloud computing environment. ScienceDirect / *Journal Name*.
- [7] R. A. Chandra, Murhaban, Suryadi, dan Mukhlizar, “Analisis dan Perbandingan Kinerja Proxmox Virtual Environment dalam Virtualisasi pada OS Debian dan Ubuntu,” *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 8, no. 3, Juni 2024.
- [8] S. Sheila, “Evaluasi Teknologi Virtualisasi Mesin Proxmox untuk Mempersiapkan Infrastruktur Server,” *SINTESIA: Jurnal Sistem dan Teknologi Informasi Indonesia*, vol. 4, no. 1, pp. 10–13, Agustus 2024
- [9] S. Ardima dan Wisnu, “Otomasi Manajemen dan Pengawasan Linux Container,” *Jurnal Teknologi Informasi dan Sistem Komputer*, vol. 10, no. 2, pp. 45–52, 2023.
- [10] R. Oliveira, N. Laranjeiro, dan M. Vieira, “A composed approach for automatic classification of web services robustness,” dalam *Proc. IEEE Int. Conf. on Software Quality, Reliability and Security Companion*, 2017, pp. 622–623
- [11] A. Nugroho dan R. Hidayat, “Implementasi Virtualisasi Server Menggunakan Proxmox VE untuk Optimasi Sumber Daya,” *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIK)*, vol. 9, no. 2, pp. 215–222, 2022.
- [12] R. Saputro dan H. Anwar, “Virtualisasi Infrastruktur Jaringan Menggunakan Proxmox VE,” *Jurnal Informatika Mulawarman*, vol. 15, no. 2, pp. 98–105, 2021.
- [13] J. Simonsson, L. Zhang, B. Morin, B. Baudry, dan M. Monperrus, “Observability and Chaos Engineering on System Calls for Containerized Applications in Docker,” *arXiv preprint*, Jul. 2019.
- [14] Jody, “Performance Evaluation of Cloud-Init as Deployment Automation, Virtual Machine, and LXC Container on Proxmox VE for AI LLM Deployment,” *SISFOKOM*, vol. 15, no. 01, Dec. 2025
- [15] Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, “Basic concepts and taxonomy of dependable and secure computing,” *IEEE Trans. Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004.
- [16] A. Wibowo dan N. Lestari, “Monitoring Penggunaan Sumber Daya Server Virtual Berbasis Linux,” *Jurnal Komputer dan Aplikasi*, vol. 9, no. 4, pp. 310–318, 2023.
- [17] C. Pahl, A. Brogi, P. Jamshidi, and J. Soldani, “Cloud container technologies: A state-of-the-art review,” *IEEE Trans. Cloud Computing*, 2019.

JSI : Jurnal Sistem Informasi (*E-Journal*)

ISSN Print : 2085-1588

ISSN Online : 2355-4614

LINK : <https://jsi.ejournal.unsri.ac.id/index.php/jsi/index>

- [18] M. Villamizar et al., “Evaluating the Monolithic and the Microservice Architecture Pattern to Deploy Web Applications in the Cloud,” *Proc. IEEE Int. Conf. on Computing, Networking and Communications (ICNC)*, 2015, pp. 583–590.
- [19] R. Ananda, F. Hidayat, dan D. Kurniawan, “Evaluasi Ketersediaan Layanan Web Server pada Lingkungan Virtualisasi,” *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, vol. 7, no. 3, pp. 512–519, 2023.
- [20] I. Firmansyah dan B. Pratama, “Studi Ketahanan Sistem Server terhadap Gangguan Menggunakan Pendekatan Fault Injection,” *Prosiding Seminar Nasional Informatika*, pp. 210–216, 2022.
- [21] A. Bhardwaj, “Virtualization in cloud computing: Moving from hypervisor to containerization—A survey,” *Journal of Cloud Computing*, 2021.
- [22] Wijayanto, “Rancang Bangun Private Server Menggunakan Platform Proxmox dengan Studi Kasus PT. MKNT,” *Journal ICTEE*, vol. 2, no. 2, pp. 41–49, 2024. .
- [23] R. Buyya, C. S. Yeo, dan S. Venugopal, “Market-Oriented Cloud Computing: Vision, Hype, and Reality,” *Proc. IEEE Int. Conf. on High Performance Computing and Communications*, 2008, pp. 5–13.
- [24] J. Gray, “Why Do Computers Stop and What Can Be Done About It?” *Tandem Technical Report*, vol. 85, no. 7, pp. 1–36, 1985.
- [25] Soni, Y. P. Magister, dan B. Sugiantoro, “Teknik Akuisisi Virtualisasi Server Menggunakan Metode Live Forensic,” *Teknomatika: Jurnal Informatika dan Komputer*, vol. 1, no. 1, pp. 12–20, 2025.